



Universidad Autónoma de Baja California (UABC)

Facultad de Ciencias de la Ingeniería y Tecnología (FCITEC)

Ingeniería en Desarrollo de Software

Proyecto Final

Monitor Cardíaco IoT con ESP32 y MAX30102

Asignatura: Organización de Computadoras

Semestre: 2026-1

Grupo: Grupo 534

Profesor: M.C. Jose Manuel Muñoz Contreras

Alumno: Reyes Gonzalez Arath — 1293182

Valle de las Palmas, Baja California, 13 de mayo de 2026

Resumen

En este proyecto se diseñó e implementó un monitor cardíaco portátil basado en el microcontrolador ESP32 [1] y el sensor óptico MAX30102 [2], capaz de medir la frecuencia cardíaca (BPM) y la saturación de oxígeno en sangre (SpO2) mediante fotoplethysmografía [3]. El sistema muestra los resultados en tiempo real sobre una pantalla OLED SSD1306 [4] mediante el protocolo I2C [5], y transmite los datos hacia Google Sheets a través de WiFi y un Google Apps Script [6]. Adicionalmente, el ESP32 actúa como servidor web embebido que expone un dashboard accesible desde cualquier dispositivo en la misma red [7]. El dispositivo evolucionó desde un prototipo en protoboard hasta una versión portátil alimentada por una batería LiPo de 3.7V, un módulo cargador TP4056 [8] y un convertidor elevador MT3608 [9] que suministra 5V regulados al ESP32.

Abstract

This project involved the design and implementation of a portable cardiac monitor based on the ESP32 microcontroller and the MAX30102 optical sensor, capable of measuring heart rate (BPM) and blood oxygen saturation (SpO2) through photoplethysmography. The system displays real-time results on an SSD1306 OLED screen via the I2C protocol and transmits data to Google Sheets through WiFi and a Google Apps Script. Additionally, the ESP32 acts as an embedded web server exposing a real-time dashboard. The device evolved from a breadboard prototype to a portable version powered by a 3.7V LiPo battery, a TP4056 USB-C charging module, and an MT3608 boost converter supplying 5V to the ESP32.

Palabras clave: MAX30102, ESP32, IoT, BPM, SpO2, OLED, I2C, Google Sheets, LiPo.

Tabla de Contenido

1. Introducción	4
2. Objetivos	4
2.1. Objetivo general	4
2.2. Objetivos específicos	4
2.3. Competencias a desarrollar	5
3. Marco Teórico	5
3.1. Microcontrolador ESP32 y configuración de pines GPIO	5
3.2. Sensor MAX30102 — Fotoplethysmografía	6
3.3. Protocolo I2C y justificación de su uso	6
3.4. Pantalla OLED SSD1306	6
3.5. Sistema de alimentación portátil	6
3.6. Backend IoT: Google Apps Script y servidor web embebido	7
4. Materiales y Equipos	7
4.1. Hardware	7
4.2. Software y servicios	7
5. Diseño del Sistema	8
5.1. Arquitectura general	8
5.2. Diagrama de conexión del circuito	8
5.3. Justificación de la selección de pines y módulos	9
6. Metodología y Desarrollo	9
6.1. Fase 1 — Primeros prototipos en protoboard	9
6.2. Fase 2 — Configuración del entorno de desarrollo	10
6.3. Fase 3 — Ajuste iterativo del firmware	10
6.4. Fase 4 — Máquina de estados	10
6.5. Fase 5 — Google Sheets y servidor web embebido	11
6.6. Fase 6 — Pruebas de conexiones	12
6.7. Fase 7 — Sistema de alimentación portátil	12
6.8. Fase 8 — Soldadura en perfboard	13
7. Resultados y Discusión	14
7.1. Prototipo final	14
7.2. Resultados de medición	15
7.3. Análisis y discusión	16

8. Problemas Encontrados y Soluciones	17
9. Conclusión	17

1. Introducción

El monitoreo de signos vitales ha sido históricamente una tarea restringida al ámbito clínico, requiriendo equipos costosos y personal especializado. Sin embargo, la convergencia entre microcontroladores de bajo costo con conectividad inalámbrica integrada, sensores ópticos miniaturizados y plataformas de almacenamiento en la nube ha democratizado el acceso a esta tecnología, permitiendo la construcción de dispositivos médicos orientados al Internet de las Cosas (IoT).

El presente proyecto integra el microcontrolador ESP32 [1] con el sensor MAX30102 [2], un dispositivo de fotoplethysmografía (PPG) que utiliza LEDs infrarrojos y rojos para detectar variaciones en la absorción de luz producidas por el flujo sanguíneo [3]. Esta técnica permite estimar de forma no invasiva tanto la frecuencia cardíaca como la saturación de oxígeno en sangre periférica (SpO₂), dos indicadores fundamentales del estado cardiovascular y respiratorio del paciente.

Desde la perspectiva de la *Organización de Computadoras*, el proyecto ilustra cómo los principios arquitectónicos de un sistema embebido —gestión de buses I²C [5], [10], control de periféricos GPIO [11], polling y comunicación en red— se materializan en un sistema funcional. El ciclo de desarrollo, desde la validación en protoboard hasta un dispositivo portátil con servidor web embebido [7], reproduce fielmente el proceso de ingeniería de hardware embebido en proyectos reales.

2. Objetivos

2.1 Objetivo general

Disenar, implementar y validar un sistema embebido portátil de monitoreo cardíaco no invasivo que, mediante el microcontrolador ESP32 [1] y el sensor MAX30102 [2], adquiera señales de fotoplethysmografía [3] para calcular la frecuencia cardíaca promedio y la SpO₂, visualice los resultados en una pantalla OLED SSD1306 [4] en tiempo real y los transmita a Google Sheets [6] mediante WiFi; superando los retos de soldadura confiable, calibración iterativa del firmware [12] e integración del sistema de alimentación portátil [8], [9].

2.2 Objetivos específicos

- Configurar Arduino IDE 2.3.8 con el soporte ESP32 [7] y las librerías Adafruit_SSD1306 [13], Adafruit_GFX [14], MAX30105 [12] y ArduinoJson [15].

- Establecer comunicación I2C [5] usando GPIO21 (SDA) y GPIO22 (SCL) [10], [11].
- Ajustar iterativamente el algoritmo `checkForBeat` y cálculo de SpO2 [2], [12].
- Diseñar una máquina de estados con cuatro fases: espera, medición de 60 s, presentación y transmisión de datos.
- Integrar WiFi para Google Sheets [6] y dashboard web embebido [7].
- Evolucionar el prototipo a dispositivo portátil con batería LiPo, TP4056 [8] y MT3608 [9].

2.3 Competencias a desarrollar

- Aplicar principios de arquitectura de computadoras en sistemas embebidos con bus I2C [5].
- Integrar hardware y software para adquisición, procesamiento y transmisión de señales biomédicas en IoT.
- Resolver problemas prácticos de fabricación: soldaduras, distribución de potencia y selección de componentes.
- Documentar técnicamente el proceso iterativo de desarrollo hasta la versión portátil.

3. Marco Teórico

3.1 Microcontrolador ESP32 y configuración de pines GPIO

El ESP32 es un sistema en chip (SoC) desarrollado por Espressif Systems que integra un procesador Xtensa LX6 de doble núcleo a 240 MHz, 520 KB de SRAM, conectividad WiFi IEEE 802.11 b/g/n y Bluetooth 4.2 [1]. Dispone de 34 pines GPIO programables con funciones multiplexadas que incluyen I2C, SPI, UART, ADC y PWM [11].

Para este proyecto se utilizaron GPIO21 y GPIO22 como bus I2C principal. Esta elección no es arbitraria: el *Technical Reference Manual* del ESP32 [16] establece que estos pines son los predeterminados para el periférico I2C 0, lo que significa que el *framework* de Arduino para ESP32 [7] los inicializa automáticamente con `Wire.begin()` sin necesidad de especificar pines adicionales, reduciendo conflictos con otras funciones del chip. El pin VIN acepta entre 4.5V y 5.5V y alimenta el regulador lineal interno que genera los 3.3V para el núcleo y los periféricos [1].

3.2 Sensor MAX30102 — Fotopletismografía

El MAX30102 es un módulo integrado de oximetría de pulso y monitoreo de frecuencia cardíaca [2]. Incorpora dos LEDs (rojo a 660 nm e infrarrojo a 880 nm), un fotodetector de alta sensibilidad y electrónica de acondicionamiento en un encapsulado de 5.6 mm × 3.3 mm. La comunicación es por I2C en la dirección fija 0x57.

El principio se basa en la fotopletismografía (PPG), técnica no invasiva que mide variaciones en el volumen sanguíneo mediante absorción diferencial de luz [3]. La librería MAX30105 de SparkFun [12], disponible en GitHub, implementa el algoritmo `maxim_heart_rate_and_oxygen_saturation()` para calcular SpO2 y `checkForBeat()` para detección de latidos.

3.3 Protocolo I2C y justificación de su uso

El bus I2C (*Inter-Integrated Circuit*) es un protocolo serie síncrono de dos hilos — SDA (datos) y SCL (reloj)— desarrollado originalmente por Philips Semiconductors [5]. Permite conectar hasta 128 dispositivos en el mismo bus, cada uno identificado por una dirección única de 7 bits.

La elección de I2C sobre SPI se justifica por dos razones: reduce a dos las líneas de señal necesarias independientemente del número de dispositivos, simplificando el cableado en la perfboard; y el ESP32 asigna GPIO21/GPIO22 al bus I2C 0 por defecto [10], eliminando configuración adicional. En este proyecto, la OLED opera en 0x3C y el MAX30102 en 0x57, coexistiendo sin interferencia [2], [4].

3.4 Pantalla OLED SSD1306

La pantalla OLED SSD1306 de 128 × 64 píxeles [4] opera con consumo ultrabajo (típicamente menor a 20 mA), adecuado para dispositivos alimentados por batería. Se gestiona mediante `Adafruit_SSD1306` [13] y `Adafruit_GFX` [14], que abstraen operaciones gráficas sobre un buffer en RAM volcado con `display.display()`.

3.5 Sistema de alimentación portátil

El TP4056 gestiona la carga de baterías LiPo en modo CC/CV, aceptando entrada USB-C de 4.5V a 5.5V [8]. Su salida directa (3.2V–4.2V) es insuficiente para el VIN del ESP32 que requiere mínimo 4.5V [1]. El convertidor MT3608 [9] eleva este voltaje a 5.0V estables, configurados mediante potenciómetro, garantizando operación correcta incluso con batería parcialmente descargada.

3.6 Backend IoT: Google Apps Script y servidor web embebido

Google Apps Script [6] permite desplegar un endpoint HTTP público con `doPost()` que recibe JSON del ESP32 y escribe en Google Sheets, eliminando la necesidad de un servidor propio. La serialización de datos usa `ArduinoJson` [15], estándar de facto para JSON en entornos Arduino con memoria limitada. La clase `WebServer` del core ESP32 [7] sirve páginas HTML desde `PROGMEM`, accesibles desde cualquier navegador en la red local.

4. Materiales y Equipos

4.1 Hardware

- Microcontrolador: ESP32 DevKit V1 de 38 pines [1]
- Sensor cardíaco: MAX30102 [2]
- Pantalla OLED SSD1306, 128×64 píxeles, I2C [4]
- Cargador LiPo: TP4056 con conector USB-C [8]
- Convertidor elevador: MT3608 *boost converter* [9]
- Batería LiPo: 3.7V, 400 mAh
- Interruptor deslizante SPDT de 3 pines
- Perfboard, cables jumper, estano 60/40, cautín de punta fina
- Multímetro Fluke 87 True RMS

4.2 Software y servicios

- Arduino IDE 2.3.8 con soporte ESP32 [7]
- Adafruit_SSD1306 [13] y Adafruit_GFX [14]
- MAX30105 SparkFun [12]
- ArduinoJson v6 [15]
- Google Sheets y Google Apps Script [6]

5. Diseño del Sistema

5.1 Arquitectura general

La arquitectura sigue un modelo de tres capas: adquisición (MAX30102 [2]), procesamiento y visualización (ESP32 [1] + OLED [4]) y transmisión (Google Sheets [6] + dashboard web [7]). La Figura 1 ilustra este flujo.

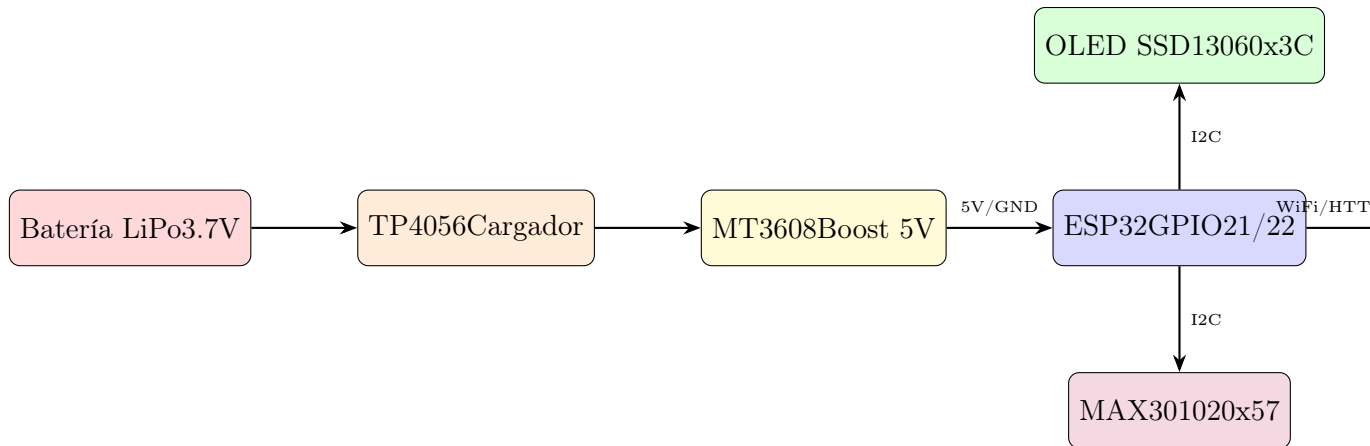


Figura 1: Arquitectura general del sistema monitor cardíaco IoT.

5.2 Diagrama de conexión del circuito

El bus I2C compartido en GPIO21/GPIO22 [10] permite la coexistencia de OLED (0x3C) y MAX30102 (0x57) [5]. El interruptor SPDT interrumpe únicamente el positivo del TP4056, manteniendo la batería siempre conectada al cargador.

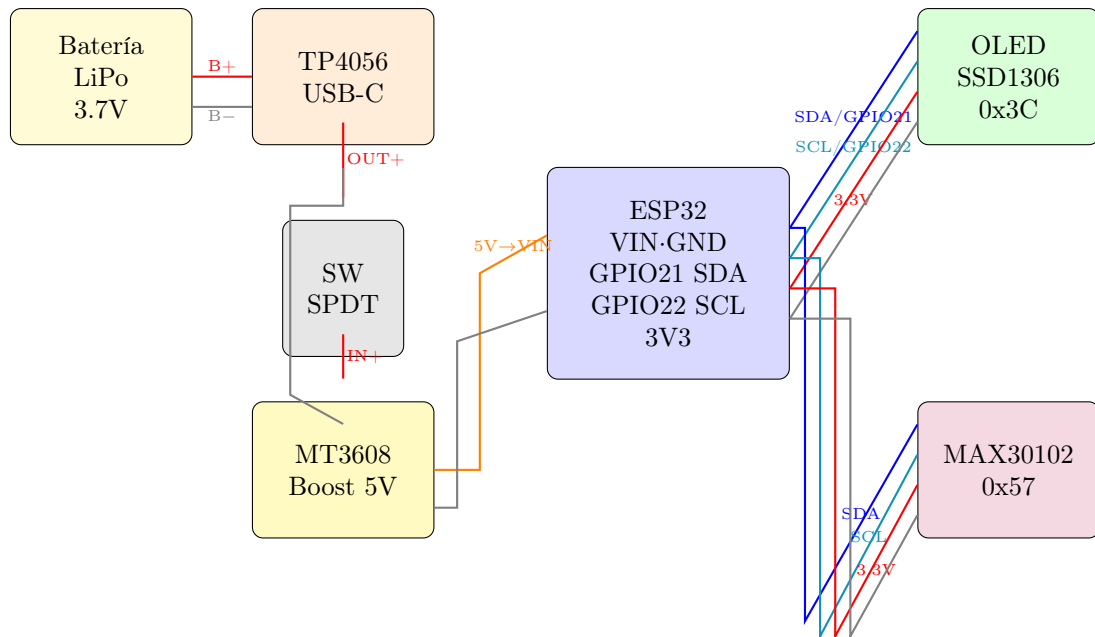


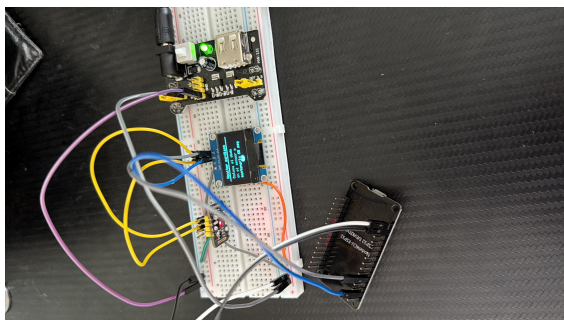
Figura 2: Diagrama completo de conexión del sistema monitor cardíaco portátil.

5.3 Justificación de la selección de pines y módulos

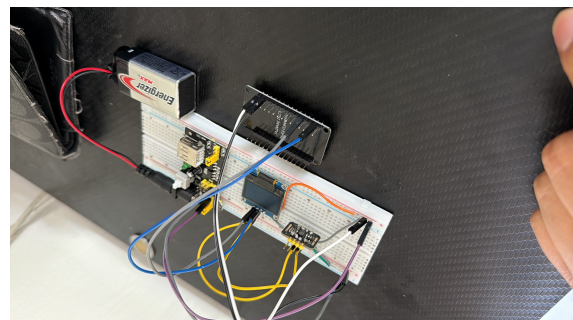
Los pines GPIO21 y GPIO22 se seleccionaron porque el ESP32 los asigna al bus I2C 0 de forma predeterminada [10], [16], evitando conflictos con otras funciones multiplexadas [11] y simplificando la inicialización con `Wire.begin(21, 22)`. Los periféricos se alimentan desde el pin 3.3V del ESP32, no directamente desde la batería, porque el regulador interno del chip garantiza voltaje estable independientemente del nivel de carga, crítico para la calibración óptica del MAX30102 [2], [3].

6. Metodología y Desarrollo

6.1 Fase 1 — Primeros prototipos en protoboard



(a) Primera conexión en protoboard.



(b) Verificación del funcionamiento inicial.

Figura 3: Primeros prototipos funcionales en protoboard.

6.2 Fase 2 — Configuración del entorno de desarrollo

1. Instalacion del soporte ESP32 en Arduino IDE 2.3.8 [7].
2. Instalacion de librerias: Adafruit_SSD1306 [13], Adafruit_GFX [14], MAX30105 [12], ArduinoJson [15].
3. Escaneo I2C confirmo 0x3C (OLED) [4] y 0x57 (MAX30102) [2].
4. Primer upload con boton BOOT en modo de descarga manual.

6.3 Fase 3 — Ajuste iterativo del firmware

El firmware original llenaba el buffer sin dedo en `setup()`, produciendo BPM = 0. La solucion fue `calibrarSensor()` que espera IR > 30,000 [12].

Cuadro 1: Evolución de la configuración del sensor MAX30102 [2].

Parámetro	Inicial	Final	Justificación
Brillo LED	60	25	IR saturaba; sin deteccion de picos
Sample average	4	4	Con 8, la senal PPG se aplanaba [3]
Sample rate	100 Hz	200 Hz	Mayor resolucion temporal
Umbral IR min.	50,000	30,000	Detecta dedo con menor presion
Umbral IR max.	—	200,000	Evita saturacion por compresion excesiva
Rango BPM	20–255	40–180	Filtra valores fuera del rango fisiologico

6.4 Fase 4 — Máquina de estados

```
1  enum Estado { ESPERANDO, MIDIENDO, MOSTRANDO_RESULTADO,
2      SUBIENDO };
3  void loop() {
4      server.handleClient(); // atiende peticiones web
5      long irValue = particleSensor.getIR();
6      bool dedoPresente = (irValue > 30000 && irValue < 200000);
7
8      switch (estadoActual) {
9          case ESPERANDO:
10             mostrarEsperando();
11             if (dedoPresente) { inicioMedicion=millis();
12                 estadoActual=MIDIENDO; }
13             break;
```

```
13     case MIDIENDO:
14         if (!dedoPresente) { estadoActual=ESPERANDO; break; }
15         if (millis()-inicioMedicion >= 60000)
16             estadoActual=MOSTRANDO_RESULTADO;
17         break;
18     case MOSTRANDO_RESULTADO:
19         mostrarResultado(); delay(4000); estadoActual=SUBIENDO;
20         break;
21     case SUBIENDO:
22         if (wifiOk && beatAvg>0) subirDatos(beatAvg, spo2Value);
23         estadoActual=ESPERANDO; break;
24     }
25 }
```

Listing 1: Estructura de la máquina de estados del firmware.

6.5 Fase 5 — Google Sheets y servidor web embebido

Se configuro Apps Script [6] como endpoint HTTP publico. El ESP32 serializa con ArduinoJson [15] y envia por HTTP POST:

```
1 StaticJsonDocument<256> doc;
2 doc["nombre"] = "Carmen";
3 doc["baja"] = bpmMin;
4 doc["alta"] = bpmMax;
5 doc["promedio"] = promedio;
6 doc["spo2"] = spo2;
7 String body;
8 serializeJson(doc, body);
9 http.POST(body);
```

Listing 2: Serialización JSON y envío HTTP POST.

La clase `WebServer` [7] sirve el dashboard HTML desde PROGMEM, accesible en `http://{IP_ESP32}/`.

6.6 Fase 6 — Pruebas de conexiones

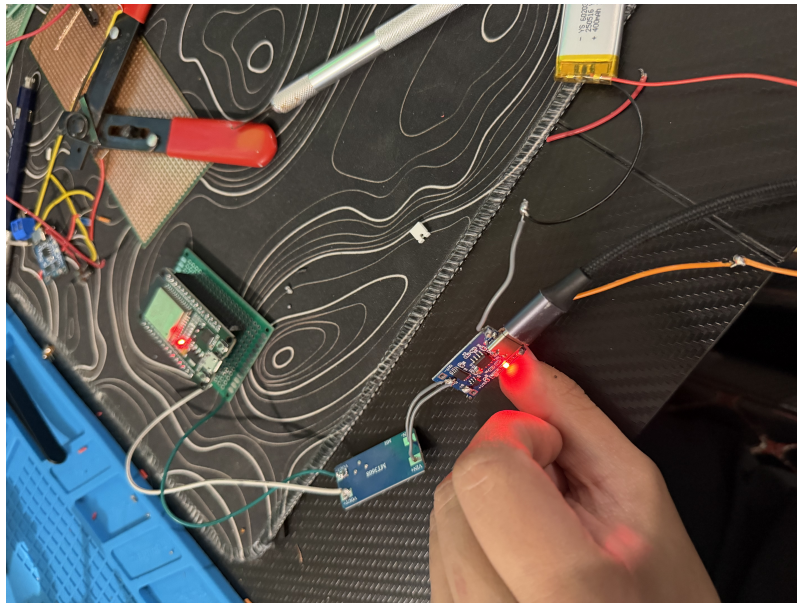


Figura 4: Verificación de conexiones con multímetro Fluke 87.

6.7 Fase 7 — Sistema de alimentación portátil



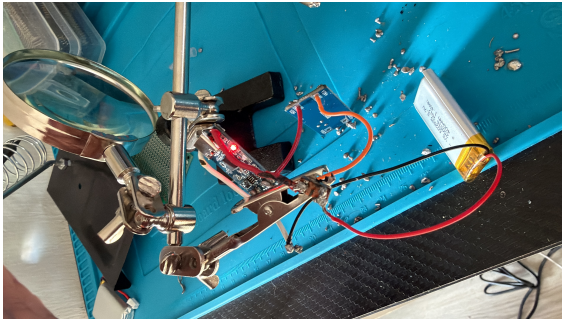
(a) Batería LiPo con TP4056 [8].



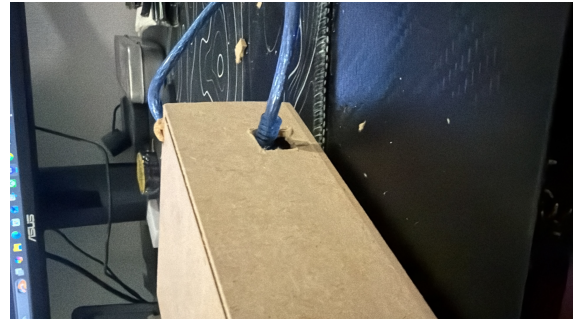
(b) Convertidor MT3608 [9] a 5V.

Figura 5: Sistema de alimentación portátil.

6.8 Fase 8 — Soldadura en perfboard



(a) Proceso de soldadura en perfboard.



(b) Detalles del ensamblado final.

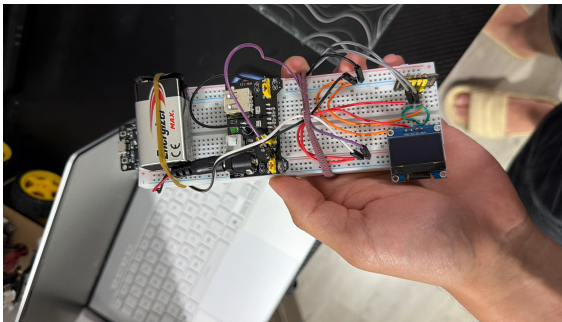
Figura 6: Etapa de soldadura y ensamblado en perfboard.

Principales retos de esta fase:

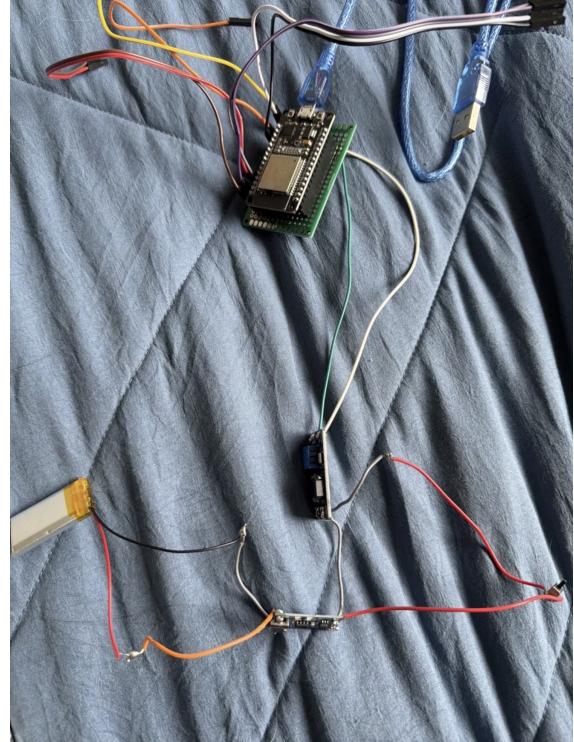
- **Soldaduras frias:** uniones opacas y granuladas refundidas hasta acabado brillante con verificación de continuidad.
- **Cortocircuito inicial:** VCC-GND conectados erróneamente; diagnosticado con multímetro ($0\ \Omega$) y corregido.
- **Voltaje insuficiente:** TP4056 [8] directo al VIN entregaba máximo 4.2V; MT3608 [9] a 5.0V lo resolvió.
- **Interruptor mal ubicado:** reubicado entre salida del TP4056 y entrada del MT3608.

7. Resultados y Discusión

7.1 Prototipo final



(a) Vista frontal del prototipo final.



(b) Vista lateral del ensamblado.

Figura 7: Prototipo final portátil del monitor cardíaco.

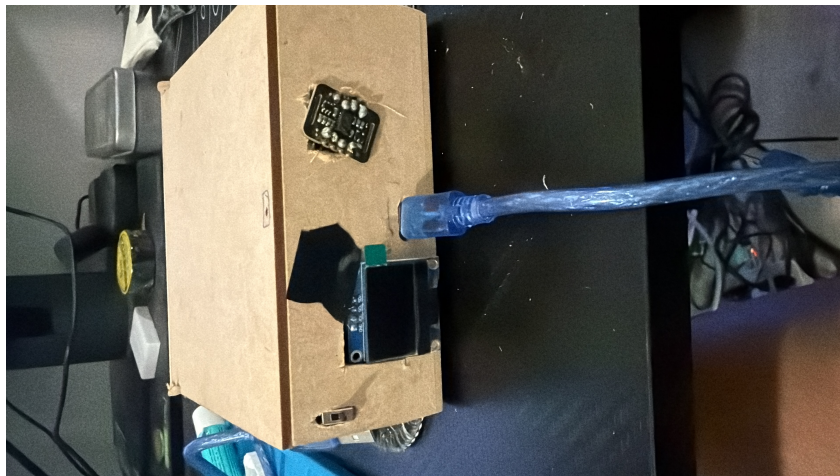
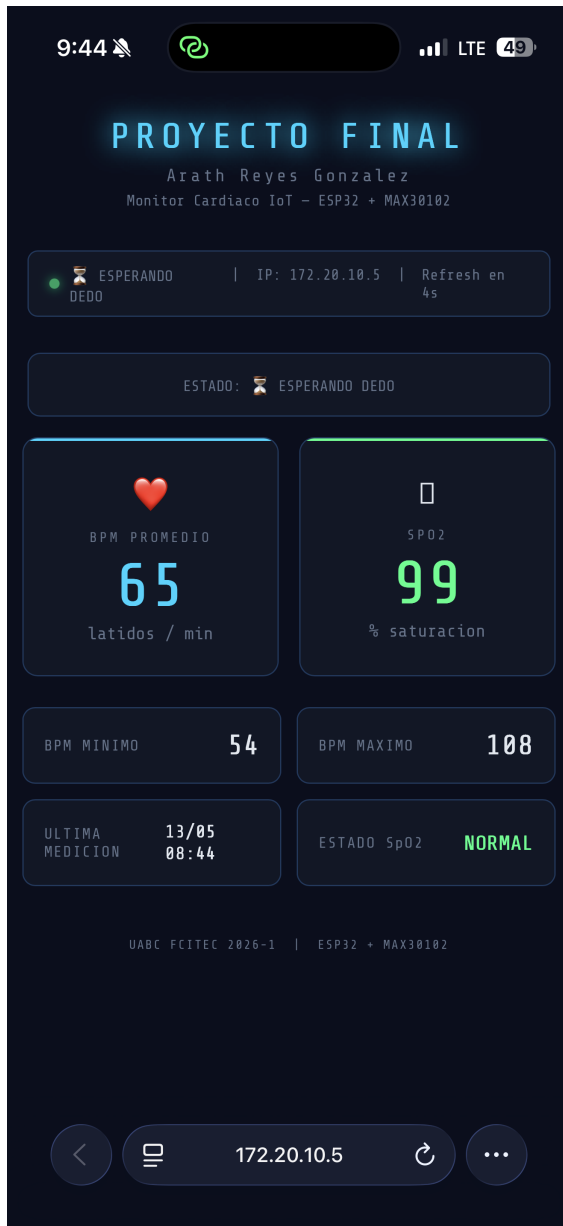
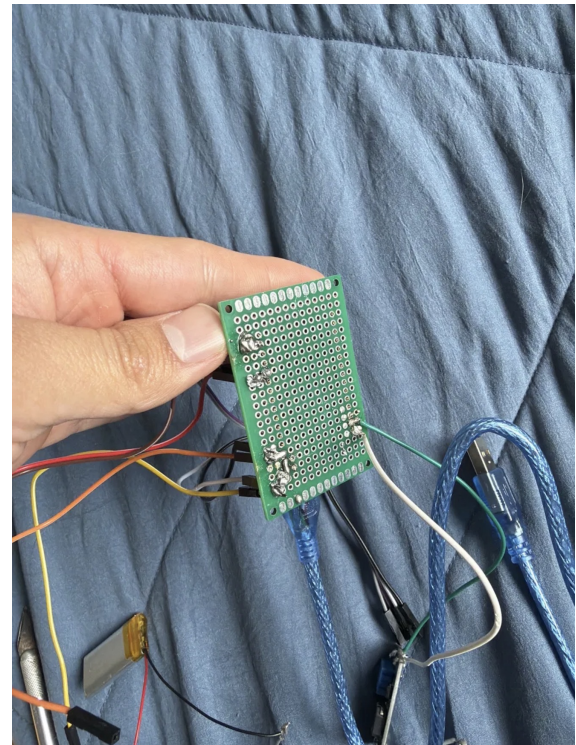


Figura 8: Proyecto terminado con todos los módulos integrados.

7.2 Resultados de medición



(a) Resultado en pantalla OLED [4].



(b) Prueba final del prototipo.

Figura 9: Resultados de medición en pantalla OLED.

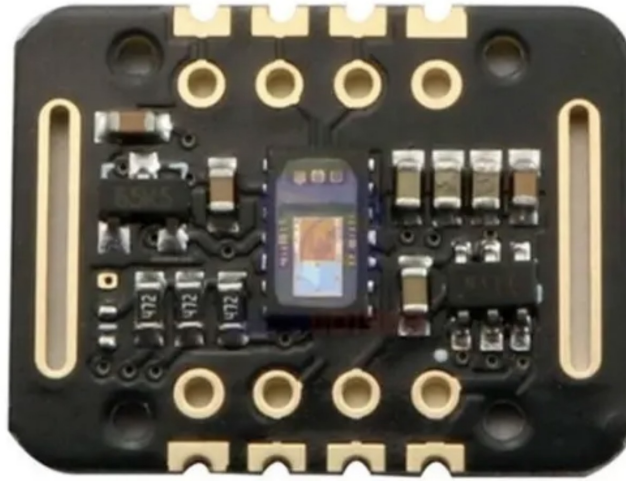


Figura 10: Sensor MAX30102 [2] en operación.

El sistema detecta latidos en 95–115 BPM y SpO₂ de 95–97 %, dentro del rango de normalidad clínica (95–100 %) [3]. La pantalla OLED [4] muestra correctamente la cuenta regresiva de 60 segundos con barra de progreso. El ESP32 [1] se conecta a la red WiFi 2.4 GHz y envía datos a Sheets [6] con respuesta HTTP 200. El servidor web [7] respondió en `http://{IP}/` con el dashboard actualizado por fetch JSON.

7.3 Análisis y discusión

1. La estabilidad mecánica del contacto del dedo fue el factor más crítico. Movimientos menores a 1 mm introducen artefactos en la señal PPG [3] que el algoritmo [12] puede interpretar erróneamente.
2. El cálculo de SpO₂ [2] requiere 100 valores de buffer, implicando aproximadamente 0.5 s de latencia inicial.
3. La batería de 400 mAh ofrece autonomía de 2–3 horas con WiFi y display activos.
4. Google Apps Script [6] presenta tiempo de respuesta variable (2–8 s) y ausencia de autenticación robusta, aceptables para este proyecto académico.

8. Problemas Encontrados y Soluciones

Cuadro 2: Registro de problemas encontrados y soluciones aplicadas.

Problema	Causa identificada	Solución aplicada
Error de boot al cargar	ESP32 no entra en modo de carga [7]	Mantener BOOT presionado al ver “Connecting...”
BPM siempre 0	Buffer llenado sin dedo en <code>setup()</code>	<code>calibrarSensor()</code> espera IR > 30,000 [12]
SpO2 = -999	LED brillante; IR saturado en 200,000	Brillo 60→25; umbral IR max. 200,000 [2]
WiFi status 6	Red 5 GHz incompatible [1]	Cambiar a red 2.4 GHz
ESP32 sobrecalentado	Cortocircuito VCC-GND en perfboard	Revision de soldaduras; verificación con multímetro
Voltaje insuficiente en VIN	TP4056 [8] directo: max. 4.2V	MT3608 [9] configurado a 5.0V
HTTP 400 desde Sheets	Script sin acceso público [6]	Reconfigurar a “Cualquier usuario”
Soldaduras frías	Temperatura insuficiente del cautín	Refundir hasta acabado brillante

9. Conclusión

El desarrollo del monitor cardíaco portátil basado en ESP32 [1] y MAX30102 [2] demuestra que la construcción de un dispositivo IoT funcional implica no solo el dominio del software embebido, sino un proceso iterativo de resolución de problemas de hardware que abarca desde la correcta selección de librerías de código abierto [12], [13], [15] hasta la ejecución confiable de soldaduras en perfboard. El firmware requirió múltiples ciclos de ajuste —modificación de parámetros del sensor [2], rediseño del algoritmo de calibración [12] y reestructuración del flujo de estados— antes de producir lecturas estables de BPM y SpO2 [3]. La etapa de portabilización evidenció que las conexiones físicas son tan críticas como el software: una soldadura fría o una conexión invertida basta para detener el sistema. El resultado final es un dispositivo autónomo capaz de medir, visualizar en OLED [4], registrar en Google Sheets [6] y exponer en un dashboard web embebido [7] los signos vitales del usuario, constituyendo un ejemplo concreto de cómo la gestión del bus I2C [5], el control de GPIO [11], los protocolos de comunicación y la arquitectura cliente-servidor convergen en un sistema embebido real fundamentado en los principios de organización de computadoras.

Referencias Bibliográficas

- [1] Espressif Systems, *ESP32 Series Datasheet, v4.4*, [En línea]. Disponible: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. Accedido: 13-may-2026, Espressif Systems, 2023. dirección: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
- [2] Maxim Integrated, *MAX30102 High-Sensitivity Pulse Oximeter and Heart-Rate Sensor for Wearable Health*, [En línea]. Disponible: <https://datasheets.maximintegrated.com/en/ds/MAX30102.pdf>. Accedido: 13-may-2026, Maxim Integrated Products, 2018. dirección: <https://datasheets.maximintegrated.com/en/ds/MAX30102.pdf>.
- [3] J. Allen, «Photoplethysmography and its Application in Clinical Physiological Measurement,» *Physiological Measurement*, vol. 28, n.º 3, R1-R39, 2007, DOI: 10.1088/0967-3334/28/3/R01. DOI: [10.1088/0967-3334/28/3/R01](https://doi.org/10.1088/0967-3334/28/3/R01).
- [4] Solomon Systech, *SSD1306 — Advance Information: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller*, [En línea]. Disponible: <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>. Accedido: 13-may-2026, Solomon Systech Limited, 2008. dirección: <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>.
- [5] NXP Semiconductors, «UM10204: I2C-bus Specification and User Manual, Rev. 7.0,» NXP Semiconductors, inf. téc., 2021, [En línea]. Disponible: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>. Accedido: 13-may-2026. dirección: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>.
- [6] Google LLC, *Google Apps Script — Web Apps*, Google Developers Documentation, [En línea]. Disponible: <https://developers.google.com/apps-script/guides/web>. Accedido: 13-may-2026, 2024. dirección: <https://developers.google.com/apps-script/guides/web>.
- [7] Espressif Systems, *Arduino Core for the ESP32 — Arduino Framework for Espressif Chips*, GitHub, [En línea]. Disponible: <https://github.com/espressif/arduino-esp32>. Accedido: 13-may-2026, 2024. dirección: <https://github.com/espressif/arduino-esp32>.
- [8] NanJing Top Power ASIC Corp, *TP4056 — 1A Standalone Linear Li-Ion Battery Charger with Thermal Regulation*, [En línea]. Disponible: <https://dlnmh9ip6v2uc.cloudfront.net/datasheets/Prototyping/TP4056.pdf>. Accedido: 13-may-2026, NanJing Top Power ASIC Corp, 2012. dirección: <https://dlnmh9ip6v2uc.cloudfront.net/datasheets/Prototyping/TP4056.pdf>.
- [9] Aerosemi Technology, *MT3608 — High Efficiency Step Up Converter*, [En línea]. Disponible: <https://www.olimex.com/Products/Breadboarding/BB->

- PWR-3608/resources/MT3608.pdf. Accedido: 13-may-2026, Aerosemi Technology Co., Ltd., 2010. dirección: <https://www.olimex.com/Products/Breadboarding/BB-PWR-3608/resources/MT3608.pdf>.
- [10] Espressif Systems, *ESP-IDF Programming Guide — I2C Driver*, Espressif Documentation, [En línea]. Disponible: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/i2c.html>. Accedido: 13-may-2026, 2024. dirección: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/i2c.html>.
- [11] Espressif Systems, *ESP-IDF Programming Guide — GPIO and RTC GPIO*, Espressif Documentation, [En línea]. Disponible: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/gpio.html>. Accedido: 13-may-2026, 2024. dirección: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/gpio.html>.
- [12] N. Hook y SparkFun Electronics, *SparkFun MAX3010x Pulse and Proximity Sensor Library*, GitHub, [En línea]. Disponible: https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library. Accedido: 13-may-2026, 2023. dirección: https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library.
- [13] Adafruit Industries, *Adafruit SSD1306 — Arduino Library for SSD1306 Monochrome OLED Displays*, GitHub, [En línea]. Disponible: https://github.com/adafruit/Adafruit_SSD1306. Accedido: 13-may-2026, 2024. dirección: https://github.com/adafruit/Adafruit_SSD1306.
- [14] Adafruit Industries, *Adafruit GFX Library — Core Graphics Library for All Adafruit Displays*, GitHub, [En línea]. Disponible: <https://github.com/adafruit/Adafruit-GFX-Library>. Accedido: 13-may-2026, 2024. dirección: <https://github.com/adafruit/Adafruit-GFX-Library>.
- [15] B. Blanchon, *ArduinoJson — A Simple and Efficient JSON Library for Embedded C++*, GitHub, [En línea]. Disponible: <https://github.com/bblanchon/ArduinoJson>. Accedido: 13-may-2026, 2024. dirección: <https://github.com/bblanchon/ArduinoJson>.
- [16] Espressif Systems, «ESP32 Technical Reference Manual, v5.2,» Espressif Systems, inf. téc., 2023, [En línea]. Disponible: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. Accedido: 13-may-2026. dirección: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.

Declaración de Autenticidad

El alumno que suscribe declara que el presente reporte de practica fue elaborado de manera personal y original, que los resultados reportados corresponden al trabajo experimental propio, y que las fuentes consultadas han sido debidamente citadas conforme a las normas academicas establecidas.

El uso de herramientas de inteligencia artificial generativa (ChatGPT, Claude, Copilot, etc.) para la redaccion de este documento **sin declaracion explicita** constituye una falta a la integridad academica y puede derivar en las sanciones establecidas en el Reglamento Escolar de la UABC.

Reyes Gonzalez Arath — 1293182

Valle de las Palmas, Baja California, 13 de mayo de 2026